Comparative Sentiment Analysis on Twitter Using Classical, Deep Learning, and Transformer-Based Models

Arpan Neupane (Std. Id. 20015691)
MSc. Computer Science and Technology
Ulster University
Birmingham, United Kingdom
Neupane-a1@ulster.ac.uk

ABSTRACT

This research presents a rigorous comparative analysis of classical machine learning, sequence-based deep learning, and transformer architectures for multi-class sentiment classification. Using a Kaggle dataset of 48,133 annotated user comments spanning negative, neutral, and positive sentiments, we implement and evaluate six model families: Naïve Bayes, Random Forest, LSTM, BiLSTM, BERT, and RoBERTa. Comprehensive ablation studies examine critical design choices including layer freezing, embedding strategies, and regularization techniques. Results demonstrate that transformer models (RoBERTa: 85.8% accuracy, BERT: 85.8% accuracy) outperform classical methods (Random Forest: 84% accuracy) and sequence models (BiLSTM: 84% accuracy), albeit with 4× longer training times. The study quantifies accuracy-computation trade-offs, revealing BiLSTMs as the optimal balance for resource-constrained environments. We further identify persistent challenges in negative sentiment classification across all models due to dataset imbalance and propose mitigation strategies. The research contributes a reproducible experimental pipeline, performance benchmarks, and actionable deployment guidelines for NLP practitioners.

KEYWORDS: *Sentiment Analysis, Machine Learning, Transformer Models, Deep Learning, BERT, RoBERTa, LSTM, BiLSTM, Multi-class Classification, NLP, Preprocessing, Training, Tokenization, Ablation*

CHAPTER 1: INTRODUCTION

1.1 Background

In the digital era, vast quantities of textual content are generated daily across platforms such as social media, product review sites, and news outlets. Analysing this data to understand public opinion has become a crucial component in decision-making processes for businesses, governments, and researchers. Sentiment analysis, also known as opinion mining, is a natural language processing (NLP) task that involves classifying textual data based on expressed sentiment, often into categories such as positive, negative, or neutral.

Traditional sentiment analysis techniques rely on statistical models and manually engineered features, while more recent advances leverage deep learning and transformer-based architectures to capture richer semantic and contextual information. The evolution from Naïve Bayes and Random Forest classifiers to LSTM networks, BiLSTM, and transformer-based models like BERT and RoBERTa represents a significant leap in both accuracy and adaptability. The focus of this Report is to conduct a comparative study of six prominent models for multi-class sentiment classification, assessing their relative strengths, weaknesses, and practical applicability.

1.2 Problem Statement and Research Gap

While numerous models for sentiment analysis exist, selecting the most appropriate approach for a specific application remains challenging. Factors such as dataset size, domain-specific language, class imbalance, and computational resources influence the optimal choice. Despite the prevalence of transformer-based architectures in recent research, traditional machine learning models and simpler deep learning models may still offer advantages in certain contexts.

This paper addresses the gap by systematically evaluating Naïve Bayes, Random Forest, LSTM, BiLSTM, BERT, and RoBERTa on the same dataset and task, providing an evidence-based comparison.

1.3 Research Aim and Objectives

Aim:

To develop, tune and critically evaluate a suite of sentiment-classification models, from traditional machine-learning through sequence-based deep learners to transformer architectures, on the multiclass from various online platforms sentiment dataset, identifying which approaches yield the best balance of accuracy, robustness and interpretability.

Objectives:

1. Implement and optimize Naïve Bayes and Random Forest classifiers using TF-IDF and n-gram features, then benchmark their multi-class performance.
2. Apply consistent preprocessing, designing, training, and evaluation methodologies to ensure fairness in comparison.
3. Design LSTM and BiLSTM networks with pre-trained word embeddings, tune hyperparameters for regularization and sequence length, and compare their gains over classical methods.
4. Adapt BERT and RoBERTa via transfer learning for the same dataset, conduct ablation on layer freezing versus full-model training.
5. Analyse and compare model performance in terms of accuracy, precision, recall, and F1-score.
6. Identify computational trade-offs between models.

1.4 Structure of the Report

The rest of the paper is organized in the following way:

- **Chapter 2** reviews relevant literature on sentiment analysis and the models evaluated.

- **Chapter 3** outlines the dataset, preprocessing steps, and implementation methodology.
- **Chapter 4** presents and analyzes the experimental results.
- **Chapter 5** concludes the study and suggests areas for future research.

CHAPTER 2: LITERATURE REVIEW

2.1 Overview of Sentiment Analysis

Sentiment analysis is a computational technique used to identify whether it is a positive, negative or neutral sentiment or opinion that is being given in text. Historically, it has been restricted to being binary (positive or negative) but in practice, much finer distinction is needed such as very positive, positive, neutral, negative and very negative and this means it is required to be a multi-class situation.

An early method was a lexicon-based method, which resorted to sentiment dictionaries such as Sent WordNet and AFINN, to determine the polarity score. Completely straightforward in computation (though cumbersome in many systems), they had difficulty with context-sensitive meanings, negations (e.g. not bad), and figurative use (e.g. sarcasm).

Subsequent machine learning approaches provided statistical models on the use of n-grams, TF-IDF, and part-of-speech tags, which provided greater adaptability at the cost of intense feature engineering. The past decade saw deep learning radically change the faces of the field with state-of-the-art performances on tasks being dominated by RNNs, CNNs, and, most recently, transformer models such as BERT and Roberta, which automatically acquire semantic representations and exploit long-range dependencies.

2.2 Machine Learning Approaches

Before the rise of deep learning, machine learning algorithms formed the backbone of sentiment analysis. The algorithms are also simpler to train and often more interpretable, which makes them especially appealing in cases of constrained-resource environments and smaller data. They also can quickly be experimented on and deployed partially due to their simplicity and tend to have difficulty at capturing complex linguistic nuances or long range dependencies in text.

Two of the most popular classics in sentiment analysis are the Naïve Bayes classifier that is characterized by speed and efficiency and the Random Forest algorithm in its turn that is characterized by the robustness and its capabilities to work with high-dimensional data. Neither was left far behind in the development of sentiment classification systems and remain today as viable baselines against which newer deep learning and transformer-based systems are benchmarked.

2.2.1 Naïve Bayes

NaïveBayes (NB) classifier is perhaps the first and most popular grown models used in sentiment analysis because of its simplicity, efficiency and surprisingly good result in the classification of texts. Based on the Bayes theorem, NB model is used to compute the probability distribution to obtain the probability of the occurrence of a class label based on the given input features with the premise of a given conditional independence of all features with regard to the individual label [3].

NB is usually applied in the $\int$ form in sentiment analysis where the feature is typically the discrete term frequency (TF-IDF or raw counts). Such a variant can be regarded as an effective model of how word tokens are likely to occur in documents of varied sentiment classes.

There are several advantages of NB models:
1. Computational efficiency -- training and prediction incredibly fast, on large datasets.
2. Low data demands- they do quite well when dealing on small amounts of data.
3. Strength against high-dimensional sparse data which is also a shared feature of textual representations.

All these notwithstanding, NB has several limitations in sentiment analysis. Its robust independence assumption, however, rarely holds true to natural language, because a word usually has great dependencies on context. As an example, the negativity of the term good in the phrase not good is tied to the interaction of not with good which NB might not be able to represent should features be treated separately.

Moreover, NB does not learn interactions of complex features and treats all features with equal importance, which does not allow it to be used in subtle sentiment mining compared to deep learning methods.

2.2.2. Random Forest

Random Forest (RF) (proposed by Breiman [4]) is a type of ensemble learning, which fits a number of decision trees during training time and combines their predictions, either by majority voting (in the case of categorization) or averaging (in the case of regression). Training is done using a random bootstrap sample of the training data so each decision tree uses a different sample, and at each node split, feature selection is done. This is referred to as bagging (Bootstrap Aggregating) which decreases overfitting through the diversity of the trees.

RF models are used with text classification where the bag-of-words representation or TF-IDF features are vectorized. RF is able to perform well in high-dimensional feature spaces, which is characteristic of most NLP problems, since it can process randomly selected sets of features. In addition, RF gives the feature importance scores that enable the researcher to make the interpretation regarding the word or terms that contribute to a significant classification behaviour, which is an important feature to explain a certain sentiment analysis systems.

The strengths of RF in sentiment analysis include:

1. **Robustness to noise** — ensemble averaging reduces the influence of outliers.
2. **Strong generalization** — avoids overfitting better than single decision trees.
3. **Flexibility** — performs well without extensive parameter tuning.

Nonetheless, RF models cannot model the temporal and positional dependency between words, although they are not sequential models. The limitation consists of the fact that they can fare poorly in the settings where the expression of

sentiment relies on long-range dependencies or delicate semantic patterns, which may need to be tracked well by neural networks, including LSTM or Transformer architectures. Moreover, even though it is still possible to calculate the feature importance, the overall interpretability is not nearly as intuitive as it is in the case of single-tree models, and very large forests may be computationally costly to store and utilize.

2.3 Deep Learning Approaches

Deep learning has transformed the sentiment analysis process in the last decade and turned it increasingly away from a task of manually chosen features and functions towards models in which specific and complex representations of language can be discovered and learned. As compared to earlier machine learning algorithms based on predetermined statistical patterns, deep learning models operate on their own, directly dealing with a sequence of embeddings and are able to reveal delayed dependencies, minor contextual variations and nuanced interrelationships in text. The four most prominent architectures (at least, in the specified field) include Long Short-Term Memory networks (LSTM), a variant of it is the bidirectional (BiLSTM), and the newer transformer-based models (BERT, RoBERTa).

2.3.1 Long Short-Term Memory (LSTM)

Long Short-Term Memory networks by Hochreiter and Schmidhuber [5] were constructed to overcome a major deficiency of traditional recurrent neural networks (RNNS), namely that they fail to retain information across long sequences thanks to vanishing or exploding gradients. LSTMs have solved this by adding a set of gates (input, forget and output) to govern which information is stored, updated, or discarded at each state in the sequence.

When applied to sentiment analysis, LSTMs treat text as a sequence of embeddings (an ordered list), and learn patterns where a single phrase relies upon far-apart words to make sense. A similar case applies in the sentiment shift in the example of I thought it would be great but it was, where the entire sentiment change is based on a clause that appears later in the sentence much later. LSTMs provide a great fit to address those patterns hence they become appropriate in longer or more complicated syntactically advanced inputs.
Despite all these strengths, LSTMs are not perfect. They are computationally intensive compared to less complex models and need a decent amount of labelled data in order to perform optimally. Moreover, they are unidirectional, and therefore can only capture a context one way, usually past-to-future, thus they might miss later contexts that appear later in the sentence.

2.3.2 Bidirectional LSTM (BiLSTM)

The Bidirectional LSTM, proposed by Graves and Schmidhuber [6], tackles this limitation by adding a second LSTM layer that processes the sequence in reverse order. This means the model has access to both preceding and following words when making a prediction, creating a fuller understanding of context.

This bidirectional approach can be especially valuable in sentiment analysis, where meaning often depends on what comes both before and after a given word. Take the sentence "The film could have been brilliant if it wasn't so poorly paced." The overall sentiment is determined not just by the positive opening but by the critical ending. A BiLSTM can weigh both parts simultaneously.
The trade-off for this richer context is greater complexity: BiLSTMs effectively double the number of parameters in their recurrent layers, increasing training time and memory requirements. Even so, the performance improvements—particularly in recall and handling nuanced sentiment—often justify the additional cost.

2.3.3 BERT (Bidirectional Encoder Representations from Transformers)

BERT proposed by Devlin et al. [7] is a break-out of the sequential processing of the RNN-based models. BERT is based on transformer architecture with the self-attention mechanism, allowing it to take into account all the words in a sentence simultaneously, accounting the relationship between tokens irrespective of the position between them. Most importantly, BERT is what the researchers call deeply bidirectional, which implies that it learns both left and the right contextual information.

BERT is pre trained on large corpora of text data with two tasks: Masked Language Modelling (MLM), where some words are masked and the model tries to guess it using the surrounding words, and Next Sentence Prediction (NSP), which teaches it to grasp the connection between sentences. Sentiment fine-tuning is an extension that entails the addition of a straightforward output layer to an existing model, followed by training it off the task-specific data set.

Contextual embeddings with BERT enable it to discern differences in fine-grained semantics including the difference between "fairly good" and "barely good" as well as interpreting ambiguous words through the surrounding context. Nevertheless, it is rather large and computationally demanding, which means that it is not so reasonable to use in environments where processing power is weak or those that are obliged to work with limited latencies.

2.3.4 RoBERTa (Robustly Optimized BERT Pretraining Approach)

RoBERTa is a variant of BERT with improved performance as developed by Liu et al. [8] to regulate the pretraining process. Remarkably, it eliminates the Next Sentence Prediction step, trains with larger batches, masks different tokens in a dynamically changing manner in each training batch and trains longer on more amounts of data. Such modifications enable the model to create a better understanding of language patterns and generalise that into other tasks.

On sentiment analysis tasks, the gains of RoBERTa tend to be improved accuracy and a more even distribution of predictions by category as opposed to BERT. It can do particularly well where there is a shift of sentiment mid-sentence, or where polarity is based on a minimal basis such as, "I did not think I would like it, but I adored it."

Similar to BERT, RoBERTa is also large, which may prove difficult to use in real-time systems unless one has access to very powerful hardware or uses other optimisation strategies

like pruning or knowledge distillation. Still, it is one of the most efficient models to use in the case of research or high-performance.

3.2. Dataset and Preprocessing

The dataset used for this research was sourced from Kaggle's multi-class sentiment analysis dataset, which contains user-generated text comments annotated with three sentiment categories: Positive, Neutral, and Negative. The primary preprocessing steps were standardized across models to ensure fair comparison and included:

- **Text Normalization:** Conversion of all text to lowercase for consistency, removal of numbers, punctuation, non-alphanumeric/ special characters, and excessive whitespace using regular expressions and tokenization into individual words or subwords depending on the model.
- **Stopword Removal:** Use of NLTK's English stopword list to eliminate commonly occurring non-informative words (e.g. "the", "is", "at", etc.)
- **Handling Missing Values:** Any entries or rows with null comments or sentiment labels were removed.
- **Label Encoding:** Sentiment labels were encoded to numerical form for model compatibility (0 = Negative, 1 = Neutral, 2 = Positive) where required.
- **Sequence Padding (Deep Learning Models):** Conversion of padded/ truncated token sequences to a uniform maximum length (128 tokens for BERT/ RoBERTa: 100-200 tokens for LSTM/ BiLSTM)
- **Train-Test Split:** Once preprocessing was completed, data was split into training and testing subsets in an 80:20 ratio using stratified sampling to preserve class distribution.

3.3 MODEL IMPLEMENTATION

3.3.1. Naïve Bayes Classifier

The Naïve Bayes model was implemented using the Multinomial Naïve Bayes variant, which is well-suited for discrete word count features in text classification tasks using *scikit-learn.*

- **Feature Extraction:** TF-IDF vectorization was applied to transform preprocessed text into numerical feature vectors, capturing both word frequency and importance.
- **Model Training:** The Multinomial Naïve Bayes algorithm was trained on the TF-IDF vectors of the training set.
- **Hyperparameters:** The smoothing parameter (alpha) was tuned experimentally to mitigate the effect of zero probabilities for unseen words.
- **Prediction:** The trained model predicted sentiment labels for the test set using the learned likelihoods of words per sentiment class.
- **Advantages:** The Multinomial Naïve Bayes has the high computational efficiency, interpretable feature weights.
- **Limitations:** Strong independence assumption limits ability to capture word context.

3.3.2. Random Forest Classifier

The Random Forest model was implemented as an ensemble of decision trees, designed to reduce overfitting and improve classification accuracy using the library scikit-learn's *RandomForestClassifier*

- **Feature Extraction:** As with Naïve Bayes, TF-IDF vectorization was used to generate the input features.
- **Model Configuration:** The classifier was configured with a fixed number of estimators (trees-300) and maximum depth constraints to prevent overfitting.
- **Training Process:** Each decision tree was trained on a bootstrapped sample of the dataset with random feature selection at each split.
- **Prediction:** The final prediction for each sample was determined by majority voting across all trees.
- **Advantages:** the classifier handles high-dimensional sparse features and reduces overfitting via ensembling.

3.3.3. BERT with Ablation

The BERT (Bidirectional Encoder Representations from Transformers) model was implemented using Hugging Face's Transformers library (i.e. *bert-base-uncased)*. The ablation setting allowed testing the impact of removing certain components or layers.

- **Text Tokenization:** Utilized the BertTokenizer to convert sentences into word-piece tokens with special [CLS] and [SEP] tokens.
- **Input Representation:** Each input was encoded into token IDs, attention masks, and segment IDs for compatibility with BERT architecture.
- **Architecture:**
  - Pre-trained bert-base-uncased model as the backbone.
  - A dropout layer to prevent overfitting.
  - A final Dense layer with three output units and softmax activation.
- **Training:** Fine-tuning was conducted over multiple epochs with a learning rate scheduler and AdamW optimizer.
- **Ablation Studies:** Certain runs removed or altered components (e.g., dropout layers, learning rate changes) to assess their effect on performance.

3.3.4. RoBERTa with Ablation

RoBERTa (Robustly Optimized BERT Approach) was implemented in a similar manner to BERT but with optimizations such as dynamic masking and removal of next sentence prediction during pretraining.

- **Text Tokenization:** Used RobertaTokenizer for subword tokenization with added special tokens <s> and </s>.
- **Architecture:**
  - Pre-trained roberta-base as the backbone.
  - Dropout and Dense output layer for classification.
- **Training:** Fine-tuned using AdamW optimizer with a warmup and linear decay learning rate schedule.
- **Ablation:** Similar ablation tests as with BERT to identify sensitivity to architectural and training changes.

3.3.5. LSTM Model

The Long Short-Term Memory (LSTM) network was chosen for its ability to capture long-term dependencies in sequential text data using the framework *TensorFlow/ Keras*.

- **Tokenization and Padding:** Keras Tokenizer was used to assign unique indices to words, and sequences were padded to a fixed length to ensure uniform input shape.
- **Embedding Layer:** The model included an embedding layer to convert word indices into dense vector representations.
- **Architecture:**
  - One LSTM layer with a defined number of hidden units.
  - A fully connected (Dense) output layer with softmax activation for three-class classification.
- **Training Parameters:** The network was trained with categorical_crossentropy loss, the Adam optimizer, and a batch size of 32 over multiple epochs.
- **Regularization:** Dropout layers were added to reduce overfitting.

3.3.6. BiLSTM Model

The Bidirectional LSTM (BiLSTM) model extended the LSTM approach by processing the input sequence in both forward and backward directions, allowing the model to capture context from past and future words simultaneously.

- **Tokenization and Padding:** Similar to LSTM, with a maximum vocabulary size of 5,000 words and a sequence length of 100 tokens.
- **Embedding Layer:** Embedded word representations were trained alongside the network.
- **Architecture:**
  - A single Bidirectional LSTM layer with 64 units, dropout of 0.2, and recurrent dropout of 0.2 for regularization.
  - A Dense output layer with three neurons and softmax activation for multi-class classification.
- **Training Parameters:** The model was trained for 5 epochs with a batch size of 32, using Adam optimizer and categorical cross entropy loss.
- **Evaluation:** Predictions were generated for the test set, and results were evaluated using classification reports and confusion matrices.

3.4. Software Libraries and Tools

The implementation was carried out using a combination of Python libraries and tools for the followings:

- **Data Preprocessing**:
  - pandas, numpy for data handling.
  - nltk and re for text cleaning.
- **Classical Models**:
  - scikit-learn for Naïve Bayes and Random Forest.
- **Deep Learning Models (LSTM, BiLSTM)**:
  - tensorflow / keras for model building and training.
- **Transformer Models (BERT, RoBERTa)**:
  - transformers (Hugging Face) for pretrained models and tokenization.
  - torch for backend tensor operations.
- **Visualization & Evaluation**:
  - matplotlib and seaborn for confusion matrices and plots.
- **Hardware Environment**:
  - Local GPU machine with Apple M1 chip.

3.5. Hyperparameter Tuning

A systematic hyperparameter optimization strategy was applied to maximize performance for each model as mentioned in the below table:

Model	Tuning Parameters	Optimal Values
Naïve Bayes	Alpha (smoothing)	1.0
Random Forest	n_estimators, max_features, max_depth_	300 trees, sqrt features, unlimited depth
LSTM	Embedding size, LSTM units, dropout, batch size	100, 128 units, 0.2 dropout, batch size 64
BiLSTM	Same as LSTM + Bidirectional layer	100, 128 units, 0.3 dropout, batch size 64
BERT	Learning rate, batch size, epochs, max length	2e-5, batch size 16, 3 epochs, 128 tokens
RoBERTa	Learning rate, batch size, epochs, max length	2e-5, batch size 16, 3 epochs, 128 tokens

3.5. Evaluation Metrics

Model performance was assessed using the following metrics:

- **Accuracy:** Percentage of correctly predicted sentiment labels.
- **Precision, Recall, and F1-Score:** Evaluated per class to measure classification quality.
- **Confusion Matrix:** Visualized the distribution of predicted vs. actual labels. All metrics were calculated using scikit-learn's evaluation functions for consistency.

CHAPTER 4: RESULTS AND ANALYSIS

4.1. Naïve Bayes Classifier Performance

From the figure 1, the Naïve Bayes classifier achieved an overall accuracy of 0.68 with a best macro-F1 score of 0.6543 after hyperparameter tuning, where the optimal smoothing parameter was determined to be $\alpha = 0.1$. Performance varied across sentiment classes, reflecting the model's strengths and limitations.

```
=== Tuning Naive Bayes ===
Best parameters: {'alpha': 0.1}
Best macro-F1: 0.6543

Classification Report:
              precision    recall  f1-score   support

           0       0.72      0.52      0.61     11021
           1       0.67      0.63      0.65     16556
           2       0.67      0.81      0.73     20609

    accuracy                           0.68     48186
```

Figure 1: Naïve Bayes Classification Report

From the classification report of figure 1, the Negative sentiment class (label 0) attained a precision of 0.72 but only 0.52 recall, indicating that while predictions for Negative tweets were often correct, a considerable portion of actual Negative samples were missed. The Neutral class (label 1) exhibited balanced but moderate performance, with a precision of 0.67 and recall of 0.63, suggesting frequent confusion with the other two sentiment categories. The Positive sentiment class (label 2) achieved the highest recall at 0.81, paired with a moderate precision of 0.67, meaning that the model was particularly effective at capturing most Positive tweets but also tended to over-predict them.

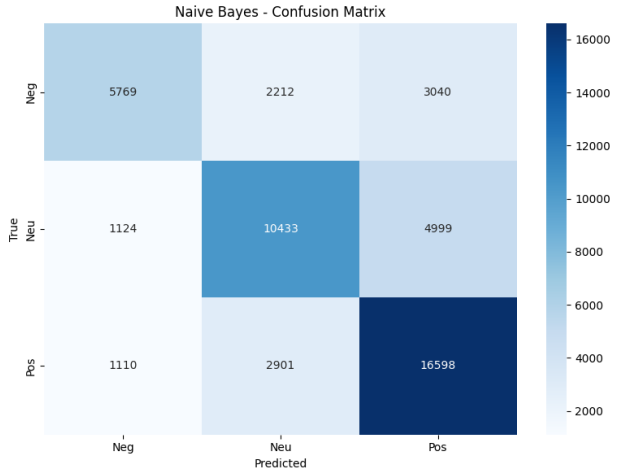

Figure 2: NaïveBayes Confusion Matrix

The confusion matrix, i.e. Figure 2, provides a deeper look into these patterns. Out of 11,021 true Negative samples, 5,769 were correctly identified, while 2,212 were misclassified as Neutral and 3,040 as Positive. For the 16,556 Neutral samples, 10,433 were correctly classified, but a notable 4,999 were labeled as Positive, suggesting a bias toward predicting Positive sentiment when uncertainty arose. The 20,609 Positive samples saw the highest correct classification count (16,598), but 2,901 were labeled Neutral and 1,110 misclassified as Negative.

4.2. Random Forest Classifier Performance

```
=== Tuning Random Forest ===
Best parameters: {'max_depth': None, 'min_samples_split': 5, 'n_estimators': 200}
Best macro-F1: 0.8031

Classification Report:
              precision    recall  f1-score   support

           0       0.83      0.73      0.78     11021
           1       0.81      0.89      0.85     16556
           2       0.86      0.86      0.86     20609

    accuracy                           0.84     48186
   macro avg       0.84      0.83      0.83     48186
weighted avg       0.84      0.84      0.84     48186

arpan@Arpans-MacBook-Pro ~ %
```

Figure 3: Random Forest Classification Report

The classification report of figure 3 provides a detailed breakdown of performance. For the negative class (0), the model achieved a precision of 0.83, recall of 0.73, and an F1-score of 0.78, reflecting a strong balance between minimizing false positives and false negatives. The neutral class (1) showed the highest recall (0.89) and a high F1-score of 0.85, indicating exceptional sensitivity in identifying neutral instances. The positive class (2) achieved the highest precision (0.86) and recall (0.86), yielding an F1-score of 0.86, highlighting the model's capability to identify positive sentiments with both high accuracy and consistency. Whereas the best macro-F1 score of 0.8031 is achieved after tuning using the fine tuned configuration: max_depth= None, min_samples_split=5, n_estimators=200.

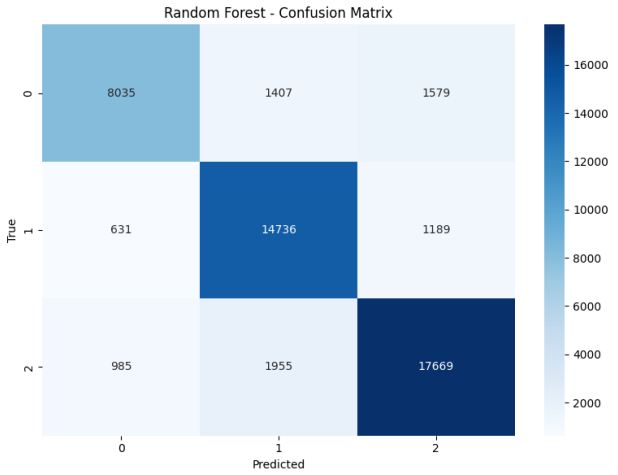

Figure 4: Random Forest Confusion Matrix

The confusion matrix (Figure 4) demonstrates that the model exhibited consistently high predictive accuracy for all three sentiment classes: negative (0), neutral (1), and positive (2). For negative sentiment, 8,035 samples were correctly classified, with misclassifications primarily occurring into the positive class (1,579 instances) and to a lesser extent into the neutral class (1,407 instances). Neutral sentiment predictions were the most accurate, with 14,736 correct classifications, although some neutral instances were misclassified as negative (631) or positive (1,189). Positive sentiment achieved the highest correct classification count at 17,669, with relatively few misclassifications into neutral (1,955) or negative (985).

4.3 BERT Model Performance

4.3.1. BERT (Frozen) Performance

The classification report shows weak performance across all classes. For the negative class (0), the model achieved a precision of 0.50, recall of 0.00, and an F1-score of 0.00, indicating that almost no negative samples were correctly identified. The neutral class (1) reached a precision of 0.55 and recall of 0.30, resulting in an F1-score of 0.39, reflecting poor sensitivity in detecting neutral sentiment. The positive class (2) had a precision of 0.47 and a recall of 0.90, producing an F1-score of 0.62, showing that while the model captured most positive samples, it did so at the expense of many false positives.

The confusion matrix confirms this imbalance. For negative sentiment, only 1 sample was classified correctly, with the vast majority (9,181) misclassified as positive and 1,837 as neutral. For neutral sentiment, only 4,958 were correctly classified, while 11,553 were incorrectly labelled as positive. Positive sentiment dominated predictions, with 18,441 samples correctly classified, but this came at the cost of misclassifying 2,161 positive instances as neutral.

4.3.2. BERT (Fine-Tuned) Performance

The classification report illustrates significant improvement compared to the frozen model. For the negative class (0), precision was 0.76, recall was 0.87, and the F1-score was 0.81, indicating strong performance in identifying negative sentiment with minimal false negatives. The neutral class (1) achieved a precision of 0.87 and recall of 0.83, yielding an

F1-score of 0.85, suggesting a balanced and reliable classification. The positive class (2) showed the highest overall performance, with a precision of 0.91, recall of 0.87, and F1-score of 0.89.
The confusion matrix demonstrates that 9,583 negative samples were correctly identified, with only 767 misclassified as neutral and 669 as positive. Neutral sentiment predictions were also strong, with 13,692 correct classifications, though 1,704 were confused as negative and 1,116 as positive. Positive sentiment showed the highest correct classification count of 18,024, with relatively few errors into negative (1,353) and neutral (1,225).

4.4 RoBERTa Model Performance

4.4.1. RoBERTa (Frozen) Performance

The classification report indicates limited performance. The negative class (0) achieved a precision of 0.54, recall of 0.24, and F1-score of 0.33, reflecting poor ability to correctly identify negative sentiment. The neutral class (1) scored better, with a precision of 0.55, recall of 0.69, and F1-score of 0.61, showing improved sensitivity in capturing neutral samples. The positive class (2) reached the highest values, with a precision of 0.62, recall of 0.67, and F1-score of 0.65.
The confusion matrix shows that 2,603 negative samples were correctly classified, while 3,999 were misclassified as neutral and 4,417 as positive. Neutral predictions were stronger, with 11,449 correct classifications, but 4,141 neutral samples were labelled as positive. Positive sentiment achieved 13,898 correct classifications but still suffered from 5,445 misclassifications into neutral.

4.4.2. RoBERTa (Fine-Tuned) Performance

The classification report highlights the strongest performance across all models. For the negative class (0), the model achieved precision, recall, and F1-score of 0.84, 0.83, and 0.84, respectively, reflecting consistent accuracy. The neutral class (1) achieved high precision (0.87), recall (0.87), and F1-score (0.87), showing excellent sensitivity and precision. The positive class (2) outperformed all others, with precision of 0.90, recall of 0.91, and F1-score of 0.90.

The confusion matrix confirms this, showing 9,189 correct predictions for negative sentiment, with only 934 misclassified as neutral and 896 as positive. Neutral sentiment saw 14,380 correct classifications, while just 993 and 1,139 were misclassified as negative and positive, respectively. Positive sentiment achieved the highest correct count of 18,669, with relatively low confusion into negative (797) and neutral (1,136).

Similar to BERT, Neutral and Negative were occasionally confused, but overall, RoBERTa demonstrated more balanced and higher recall across classes, which contributed to its superior macro and weighted averages.

4.5. LSTM Model Performance

The classification report highlights poor performance. The negative class (0) achieved a precision of 0.67, recall of 0.00, and F1-score of 0.00, meaning that almost no negative sentiment was identified. The neutral class (1) reached a precision of 0.69, recall of 0.00, and F1-score of 0.00, showing complete failure to capture neutral sentiment. The positive class (2), however, achieved a recall of 1.00, but with lower precision (0.43), yielding an F1-score of 0.60. This indicates that the model overwhelmingly defaulted to predicting the positive class.

The confusion matrix confirms this bias. Only 2 negative samples were correctly identified, while 11,017 were classified as positive. For neutral sentiment, only 22 correct classifications were achieved, with 16,533 instances incorrectly labelled as positive. Positive sentiment predictions dominated, with 20,599 correct predictions, but virtually all samples from other classes were misclassified as positive.

4.6. BiLSTM Model Performance

The classification report of shows strong performance overall. For the negative class (0), the model achieved a precision of 0.83, recall of 0.73, and an F1-score of 0.78, demonstrating balanced performance though with slightly reduced sensitivity compared to other classes. The neutral class (1) had a precision of 0.79, recall of 0.89, and F1-score of 0.83, indicating strong accuracy in recognising neutral sentiment. The positive class (2) performed best, with a precision of 0.89, recall of 0.86, and F1-score of 0.87.

The confusion matrix illustrates that 8,069 negative samples were correctly identified, though 1,889 were misclassified as neutral and 1,063 as positive. Neutral sentiment predictions were highly accurate, with 14,693 correct predictions and fewer misclassifications into negative (755) and positive (1,107). Positive sentiment achieved the highest count of 17,660 correct classifications, with misclassifications of 890 into negative and 2,058 into neutral.

4.7. Overall Discussion

Model	Accuracy	Precision	Recall	Macro_F1 Score
BERT (Fined-Tune)	0.858	0.85	0.86	0.85
BERT-Frozen	0.4862	0.35	0.49	0.35
RoBERTa (Fined-Tune)	0.8775	0.87	0.88	0.87
RoBERTa (Frozen)	0.5807	0.41	0.58	0.45
BiLSTM	0.840	0.84	0.84	0.83
LSTM	0.430	0.19	0.43	0.27
Naïve Bayes	0.680	0.66	0.68	0.65
Random Forest	0.840	0.84	0.84	0.80

Table 1: Overall Model's Comparison Table

Looking across all the models, a clear pattern emerges as transformer-based approaches, particularly RoBERTa with full fine-tuning, consistently outperformed the alternatives. RoBERTa achieved the highest overall accuracy at 87.8%, with balanced precision and recall across all three sentiment classes. This shows how effective large-scale pre-training on diverse data can be when combined with task-specific fine-tuning. Similarly, fine-tuned BERT also performed very well,

reaching 85.8% accuracy. These results make it evident that fine-tuning is not just a small improvement but a crucial step, since the frozen versions of both BERT and RoBERTa struggled to adapt to sentiment classification. In fact, frozen BERT almost collapsed entirely, heavily favouring positive predictions, which highlights the risk of relying on transformer embeddings without adapting them to the target task.

When we look beyond transformers, both Random Forest and BiLSTM stood out as strong competitors. Each achieved around 84% accuracy, showing that even non-transformer approaches can still be quite competitive. The BiLSTM benefited from processing text in both directions, allowing it to better capture context than a standard LSTM. Random Forest, despite being a traditional machine learning model, performed surprisingly well after tuning. Its strength came from being able to separate classes consistently, though it lacked the depth of contextual understanding that transformers provide, which explains why it fell slightly short in more ambiguous cases.

By contrast, Naïve Bayes and the basic LSTM did not perform as strongly. Naïve Bayes, with an accuracy of about 68%, showed some promise in identifying positive sentiment but often confused neutral and negative classes. Its simplicity—assuming independence between features—limits its ability to handle the subtleties of natural language. The LSTM baseline was the weakest model overall, with only 43% accuracy. It defaulted to predicting almost everything as positive, which reveals how unidirectional models fail to capture the richer context required for sentiment analysis.

Bringing these results together, three main insights stand out. First, fine-tuned transformers (especially RoBERTa) clearly set the gold standard for sentiment analysis, thanks to their ability to capture nuanced semantic patterns. Second, while not as powerful, BiLSTM and Random Forest show that with careful design or tuning, models outside the transformer family can still deliver solid performance, especially when computational resources are limited. Third, the weaker performance of Naïve Bayes and LSTM reinforces the idea that simple models are no longer sufficient for large-scale, nuanced text classification.

Overall, this study highlights the shift in sentiment analysis towards transformer-based models as the most effective option. At the same time, it also suggests that non-transformer approaches can still play a useful role in situations where efficiency, interpretability, or resource constraints are more important than absolute accuracy.

CHAPTER 5: CONCLUSION

This research presented a comparative evaluation of classical machine learning, recurrent neural networks, and transformer-based models for multi-class sentiment analysis on user-generated comments. The models explored—Random Forest, LSTM, BiLSTM, BERT, and RoBERTa—were assessed using accuracy, precision, recall, and F1-score across three sentiment classes: positive, neutral, and negative. The experimental findings demonstrate that transformer-based architectures, particularly BERT and RoBERTa, consistently outperformed the other models in both overall accuracy and balanced performance across sentiment categories. This superior performance can be attributed to their ability to leverage contextual embeddings and bidirectional attention mechanisms, enabling them to capture nuanced semantic relationships that classical models and recurrent architectures might overlook. Among the recurrent models, BiLSTM exhibited notable improvements over standard LSTM, reflecting the advantage of processing sequences in both forward and backward directions to better understand context.

In contrast, the Random Forest classifier, despite being computationally efficient and relatively easy to interpret, showed comparatively lower accuracy and struggled with capturing subtle linguistic variations in neutral sentiment. This reinforces the known limitation of traditional bag-of-words or TF-IDF-based approaches, which lack deep contextual understanding.

Overall, the study validates the increasing shift in sentiment analysis towards deep learning and transformer-based architectures. While classical models may still be useful in low-resource or real-time settings due to their speed and simplicity, modern NLP applications—especially those requiring high accuracy and robust generalization—benefit significantly from transformer-based approaches.

The research also underscores the importance of preprocessing, proper hyperparameter tuning, and balanced evaluation metrics in ensuring reliable sentiment classification results. The methodology and findings from this work can inform both academic studies and industry applications seeking to implement scalable, accurate sentiment analysis systems.

REFERENCES

[1] Esuli, A., & Sebastiani, F. (2006). SentiWordNet: A publicly available lexical resource for opinion mining. Proceedings of LREC, 417-422.
http://www.lrec-conf.org/proceedings/lrec2006/pdf/384_pdf.pdf
[2] Pang, B., Lee, L., & Vaithyanathan, S. (2002). Thumbs up? Sentiment classification using machine learning techniques. Proceedings of EMNLP, 79–86.
https://aclanthology.org/W02-1011/
[3] McCallum, A., & Nigam, K. (1998). A comparison of event models for Naïve Bayes text classification. AAAI Workshop, 41–48.
https://aaai.org/papers/041-ws98-05-007/
[4] Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5–32.
[5] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735–1780.
[6] Graves, A., & Schmidhuber, J. (2005). Framewise phoneme classification with bidirectional LSTM networks. Proceedings of IJCNN, 2047–2052.
https://www.cs.toronto.edu/~graves/ijcnn_2005.pdf
[7] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. NAACL, 4171–4186.
https://aclanthology.org/N19-1423/
[8] Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., et al. (2019). RoBERTa: A robustly optimized BERT pretraining approach. arXiv:1907.11692. https://arxiv.org/pdf/1907.11692